



**MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

Cimendur Rustem

FAF-243

Report

Laboratory work n.1

of Cryptography and Security

Chisinau – 2026

Caesar cipher over the Romanian alphabet

1 Short theory

The aim of this work was to implement the Caesar cipher for the Romanian alphabet and then add a keyword that changes the alphabet order. Both versions encrypt and decrypt text using a numeric shift. With the same keys, decryption restores the message in uppercase, without spaces.

Romanian alphabet and shift

The alphabet contains 31 letters. Their codes are their positions in the order given in the task, starting with A = 0, Ă = 1 and Â = 2, and ending with Z = 30. The shift k must be an integer from 1 to 30. A shift of zero is excluded because it leaves the message unchanged.

For a plaintext letter with code x , encryption uses the formula

$$c = (x + k) \bmod 31$$

For a ciphertext letter with code y , decryption uses

$$m = (y - k) \bmod 31$$

Modulo keeps the result between 0 and 30, so the shift wraps around the alphabet. For example, with $k = 3$, A becomes B, since B has code 3. Z becomes Â because 33 modulo 31 is 2. There are only 30 possible keys, so all of them can be tried to recover a readable message.

Permutation with a keyword

The second version uses a keyword as an additional key. Its letters are written first, keeping only the first occurrence of each letter. The unused Romanian letters are then added in their usual order. Encryption and decryption use the same formulas, but a letter is now indexed in this new alphabet.

There are $31!$ possible alphabet orders and 30 shifts for each order. The number of pairs of alphabet orders and shifts is therefore

$$N = 31! \times 30$$

Different key pairs can produce the same substitution, so this is not a count of distinct encryptions. The keyword makes the choice of keys much larger, but the cipher still replaces every occurrence of a letter with the same letter. Letter frequencies and repeated patterns remain visible, which makes frequency analysis possible.

2 Implementation

The implementation is a console program written in Python. It is kept in one file, `caesar_cipher.py`. Both tasks use the same function for shifting letters; only the alphabet passed to that function changes.

Alphabet and text preparation

```
ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPSŞTŢUVWXYZ"  
ALLOWED_LETTERS = ALPHABET + ALPHABET.lower()
```

This string follows the encoding table from the task. The program uses `alphabet.index(letter)` to obtain a letter code, rather than using ASCII or Unicode code points to calculate the shift. The second string allows both uppercase and lowercase input.

```
def normalize_cedilla_letters(text):  
    text = text.replace("Ş", "Ŗ").replace("ş", "ŗ")  
    text = text.replace("Ţ", "Ț").replace("ț", "ȣ")  
    return text
```

These replacements accept `Ş` and `Ţ` with a cedilla and convert them to `Ŗ` and `Ț` with a comma below. After validation, the message is converted to uppercase and ordinary spaces are removed.

```
return message.upper().replace(" ", "")
```

Input validation

```
def read_key():  
    while True:  
        value = input("Enter numeric key (1-30): ")  
        try:  
            key = int(value)  
        except ValueError:  
            print("Key must be an integer from 1 to 30.")  
            continue  
  
        if 1 <= key <= 30:  
            return key  
  
        print("Key must be an integer from 1 to 30.")
```

The input is converted to an integer and checked against the range 1 to 30. If conversion fails or the number is outside this range, the program prints the allowed values and asks for the key again.

```
for character in message:  
    if character != " " and character not in ALLOWED_LETTERS:  
        invalid_character = character  
        break
```

Each message character is checked before uppercasing. A rejected character is shown in the error message, and the whole message is requested again. The keyword is checked in the same way, but spaces are not allowed. It must contain at least seven letters. This length check is made before repeated letters are removed.

Permuted alphabet

```
def build_permuted_alphabet(keyword):
    permuted_alphabet = []

    for letter in keyword:
        if letter not in permuted_alphabet:
            permuted_alphabet.append(letter)

    for letter in ALPHABET:
        if letter not in permuted_alphabet:
            permuted_alphabet.append(letter)

    return "".join(permuted_alphabet)
```

The first loop adds keyword letters only if they are not already present. The second loop adds every missing letter from the Romanian alphabet. The result contains all 31 letters exactly once.

For the keyword *criptografie*, the distinct letters are C, R, I, P, T, O, G, A, F and E. They form the beginning of the permuted alphabet. The program prints the full order so that the substitution can be checked by hand.

```
CRIPTOGA FEĂĂBDHÎJKLMNQSSȚUVWXYZ
```

Encryption and decryption

```
def transform(message, key, alphabet, decrypt=False):
    result = ""

    for letter in message:
        position = alphabet.index(letter)
        if decrypt:
            new_position = (position - key) % len(alphabet)
        else:
            new_position = (position + key) % len(alphabet)
        result += alphabet[new_position]

    return result
```

For each letter, the function finds its position in the selected alphabet. Encryption adds the key, while decryption subtracts it. The remainder selects the new letter. In Python, taking modulo with a positive alphabet length also handles a negative position correctly, so no separate correction is needed for decryption.

For example, C has position 0 in the permuted alphabet above. With key 3, it becomes P, which has position 3. Using the same keyword and subtracting 3 maps P back to C.

Program flow

The menu first selects the cipher version and then the operation. The program reads the numeric key, asks for a keyword for the second version, and reads the message. It prints the result and returns to the menu. Choosing 0 ends the program.

3 Results

The message În țară sunt șapte câini contains all five Romanian letters with diacritics, lowercase letters and spaces. Both versions used numeric key 3; the second also used the keyword criptografie. Decrypting each result with the same keys returned the original text in uppercase without spaces.

Caesar cipher

```
Choose operation: 1 – encrypt, 2 – decrypt: 1
Enter numeric key (1–30): 3
Enter message: În țară sunt șapte câini
Result (ciphertext): LQWBTCT̃XQVUBSVHFDKQK
```

Figure 1. Console output for encryption with the ordinary alphabet.

```
Choose operation: 1 – encrypt, 2 – decrypt: 2
Enter numeric key (1–30): 3
Enter message: LQWBTCT̃XQVUBSVHFDKQK
Result (plaintext): ÎNȚARĂSUNTȘAPTECÂINI
```

Figure 2. Console output for decryption with the ordinary alphabet.

Cipher with a keyword

```
Choose operation: 1 – encrypt, 2 – decrypt: 1
Enter numeric key (1–30): 3
Enter keyword: criptografie
Permuted alphabet: CRIPTOGAFEĂÂBDHÎJKLMNQSȘȚUVWXYZ
Enter message: În țară sunt șapte câini
Result (ciphertext): LȘWĂTDUXȘAVĂGABPHOȘO
```

Figure 3. Console output for encryption with the permuted alphabet.

```
Choose operation: 1 – encrypt, 2 – decrypt: 2
Enter numeric key (1–30): 3
Enter keyword: criptografie
Permuted alphabet: CRIPTOGAFEĂÂBDHÎJKLMNQSȘȚUVWXYZ
Enter message: LȘWĂTDUXȘAVĂGABPHOȘO
Result (plaintext): ÎNȚARĂSUNTȘAPTECÂINI
```

Figure 4. Console output for decryption with the permuted alphabet.

Invalid input

The numeric inputs 0, 31 and abc were rejected. The program asked for a key again after each error. It also rejected the exclamation mark in Salut! and accepted the corrected message Salut.

```
Enter numeric key (1-30): 0
Key must be an integer from 1 to 30.
Enter numeric key (1-30): 31
Key must be an integer from 1 to 30.
Enter numeric key (1-30): abc
Key must be an integer from 1 to 30.
Enter numeric key (1-30): 3
Enter message: Salut!
Invalid character '!'. Allowed: letters A-Z, Ă, Â, Î, Ș, Ț, and spaces.
Enter message: Salut
Result (ciphertext): ȚBOXV
```

Figure 5. Rejected numeric keys and an invalid message character.

The six-letter keyword secret was rejected. The value secret! was rejected because of the exclamation mark. With the valid keyword criptografie, the program displayed the permuted alphabet and continued.

```
Enter numeric key (1-30): 3
Enter keyword: secret
Keyword must contain at least 7 Romanian letters (A-Z, Ă, Â, Î, Ș, Ț).
Enter keyword: secret!
Invalid character '!'. Allowed in the keyword: letters A-Z, Ă, Â, Î, Ș, Ț.
Enter keyword: criptografie
Permuted alphabet: CRIPTOGAFĂĂÂBDHÎJKLMNQȘȘȚUVWXYZ
Enter message: Salut
Result (ciphertext): UĂQXA
```

Figure 6. A keyword that is too short and a keyword with an invalid character.

4 Conclusions

Both versions correctly restored the prepared Romanian message. Validation rejected incorrect keys and unsupported characters, then requested new input. Using positions in the specified alphabet was necessary for the correct Romanian substitutions. The ordinary cipher has only 30 keys and can be broken by trying them all. A keyword gives more possible key choices, but each letter still has a fixed substitution. Letter frequency analysis therefore remains possible.

5 Source code

The full Python implementation is in caesar_cipher.py. Repository:

<https://github.com/fxmidaaa/cryptography-labs>

```
ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPSŞŢTUVWXYZ"
ALLOWED_LETTERS = ALPHABET + ALPHABET.lower()

def normalize_cedilla_letters(text):
    text = text.replace("Ş", "s").replace("ş", "s")
    text = text.replace("Ţ", "t").replace("ţ", "t")
    return text

def read_key():
    while True:
        value = input("Enter numeric key (1-30): ")
        try:
            key = int(value)
        except ValueError:
            print("Key must be an integer from 1 to 30.")
            continue

        if 1 <= key <= 30:
            return key

    print("Key must be an integer from 1 to 30.")

def read_message():
    while True:
        message = normalize_cedilla_letters(input("Enter message: "))
        invalid_character = None

        for character in message:
            if character != " " and character not in ALLOWED_LETTERS:
                invalid_character = character
                break

        if invalid_character is not None:
            print(
                f"Invalid character {invalid_character!r}. "
                "Allowed: letters A-Z, Ă, Â, Î, Ş, Ţ, and spaces."
            )
            continue

    return message.upper().replace(" ", "")
```

```

def read_keyword():
    while True:
        keyword = normalize_cedilla_letters(input("Enter keyword: "))
        invalid_character = None

        for character in keyword:
            if character not in ALLOWED_LETTERS:
                invalid_character = character
                break

        if invalid_character is not None:
            print(
                f"Invalid character {invalid_character!r}. "
                "Allowed in the keyword: letters A-Z, Ă, Â, Î, Ș, Ț."
            )
            continue

        keyword = keyword.upper()
        if len(keyword) < 7:
            print(
                "Keyword must contain at least 7 Romanian letters "
                "(A-Z, Ă, Â, Î, Ș, Ț)."
            )
            continue

        return keyword

def build_permuted_alphabet(keyword):
    permuted_alphabet = []

    for letter in keyword:
        if letter not in permuted_alphabet:
            permuted_alphabet.append(letter)

    for letter in ALPHABET:
        if letter not in permuted_alphabet:
            permuted_alphabet.append(letter)

    return "".join(permuted_alphabet)

def transform(message, key, alphabet, decrypt=False):
    result = ""

    for letter in message:
        position = alphabet.index(letter)
        if decrypt:
            new_position = (position - key) % len(alphabet)
        else:
            new_position = (position + key) % len(alphabet)
        result += alphabet[new_position]

    return result

```

```

def main():
    while True:
        print("\nChoose a cipher:")
        print("1 - Caesar cipher")
        print("2 - Caesar cipher with a keyword")
        print("0 - Exit")
        cipher_choice = input("Your choice: ")

        if cipher_choice == "0":
            break
        if cipher_choice not in ("1", "2"):
            print("Choose 1, 2, or 0.")
            continue

        operation = input("Choose operation: 1 - encrypt, 2 - decrypt: ")
        while operation not in ("1", "2"):
            print("Choose 1 to encrypt or 2 to decrypt.")
            operation = input("Your operation: ")

        key = read_key()
        alphabet = ALPHABET

        if cipher_choice == "2":
            keyword = read_keyword()
            alphabet = build_permuted_alphabet(keyword)
            print("Permuted alphabet:", alphabet)

        message = read_message()
        result = transform(message, key, alphabet, decrypt=(operation == "2"))

        if operation == "1":
            print("Result (ciphertext):", result)
        else:
            print("Result (plaintext):", result)

if __name__ == "__main__":
    main()

```